Next.js Architect Playbook: Production Scale at L4/L5 (7 YOE)

Overview

Next.js is a React metaframework that simplifies full-stack web development with file-based routing, SSR, API routes, and built-in optimization. At L4/L5, interviewers expect:

- Deep understanding of rendering strategies (SSR, SSG, ISR, CSR) and when each fits.
- Data fetching pattern mastery and cache invalidation reasoning.
- Performance optimization beyond defaults (bundles, images, fonts, Core Web Vitals).
- Security hardening (CSRF, XSS, secrets, middleware auth).
- Deployment and scaling strategies (Vercel, self-hosted, edge runtime).
- Production incident patterns and mitigation.

---

1) Next.js Rendering Fundamentals (Core Mental Models)

1.1 Rendering Modes and Trade-offs

Static Generation (SSG)

- Page rendered at build time and served from CDN.
- Use for: marketing sites, docs, blogs with infrequent updates.
- Pros: fastest TTFB, no server load, CDN-friendly.
- Cons: requires rebuild for content updates; not for dynamic/user-specific data.

```
// pages/blog/[slug].tsx
export const getStaticProps: GetStaticProps = async ({ params }) => {
  const post = await db.posts.findOne({ slug: params.slug });
  return {
    props: { post },
    revalidate: 3600, // ISR: revalidate every hour
  };
};

export const getStaticPaths: GetStaticPaths = async () => {
  const slugs = await db.posts.getAllSlugs();
  return {
    paths: slugs.map(slug => ({ params: { slug } })),
    fallback: 'blocking', // unknown routes SSR on first request
  };
};
```

Server-Side Rendering (SSR)

- Page rendered on every request on the server.
- Use for: personalized, user-specific, or real-time data.
- Pros: always fresh data, user-specific content.
- Cons: slower TTFB, higher server cost, cannot cache on CDN.

```tsx
// pages/dashboard.tsx
export const getServerSideProps: GetServerSideProps = async ({ req, res }) => {
  const session = await getSession({ req });
  if (!session) {
    return { redirect: { destination: '/login', permanent: false } };
  }
  const user = await db.users.findOne({ id: session.userId });
  return { props: { user } };
};
```

Incremental Static Regeneration (ISR)

- Page generated on-demand and cached; refreshes after TTL or on-demand revalidation.
- Use for: blog posts, product pages with infrequent updates.
- Pros: balance of freshness and performance; no rebuild needed.
- Cons: stale content window until revalidation; cache invalidation complexity.

```tsx
export const getStaticProps: GetStaticProps = async ({ params }) => {
  const product = await db.products.findOne({ id: params.id });
  return {
    props: { product },
    revalidate: 60, // revalidate after 60s
  };
};
```

Client-Side Rendering (CSR)

- Page shell served; content fetched and rendered on client via JS.
- Use for: interactive dashboards, real-time updates, user-specific analytics.
- Pros: highly interactive, instant user feedback, offload server.
- Cons: slower initial paint, requires client JS, SEO challenges.

```tsx
// pages/analytics.tsx
export default function Analytics() {
  const [data, setData] = useState(null);
  useEffect(() => {
    fetch('/api/analytics').then(r => r.json()).then(setData);
  }, []);
  return <div>{data ? <Chart data={data} /> : 'Loading...'}</div>;
}
```

Interview signal: "Rendering mode choice depends on audience (SEO), freshness needs, and personalization; not a one-size-fits-all."

1.2 App Router vs Pages Router (Next.js 13+)

Pages Router (traditional, still supported)

- File structure: `pages/api/users.ts`, `pages/blog/[id].tsx`.
- API routes in `pages/api/` folder.

- Data fetching via `getStaticProps`, `getServerSideProps`, `getInitialProps`.

App Router (next.js 13+, recommended)

- File structure: `app/dashboard/page.tsx`, `app/api/users/route.ts`.
- Nested layouts and segment-level data fetching.
- Server Components by default; explicit 'use client' for interactive.
- Streaming SSR and async Server Components.
- Advanced features: Middleware, Route Handlers, Intercepting Routes.

Migration considerations:

- App Router requires React 18+ and may break some legacy patterns.
- Pages Router still works but no new features; eventual deprecation.
- Phased migration: new features in App Router, legacy in Pages Router.

Interview signal: "App Router is the future; understand both for legacy codebases, but new systems should use App Router for better performance and DX."

---

2) Server Components and Client Components (App Router Deep Dive)

2.1 Server Components (Default in App Router)

- Rendered exclusively on the server.
- Direct access to databases, APIs, secrets (server-only code).
- No JS shipped to client; minimal bundle size.
- Cannot use React hooks or browser APIs (`useState`, `useEffect`, `window`).

```
// app/products/page.tsx (Server Component by default)
async function ProductsPage() {
  const products = await db.products.findMany(); // Direct DB access
  return (
    <div>
      {products.map(p => (
        <ProductCard key={p.id} product={p} />
      ))}
    </div>
  );
}
```

2.2 Client Components ('use client')

- Run in browser; can use hooks, event listeners, browser APIs.
- Re-render on client state changes.
- Still have SSR'd initial HTML; interactive after hydration.

```
'use client';

import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);
```

```
  return <button onClick={() => setCount(count + 1)}>{count}</button>;
}
```

2.3 Boundary Design (Server/Client Split)

- Keep server logic server-side; push interactivity to client boundary.
- Use 'use client' only at leaf components to minimize client bundle.

```
// app/layout.tsx (Server Component)
export default function RootLayout({ children }) {
  return (
    <html>
      <body>
        <Navbar /> {/* Server Component; no JS sent for navbar itself */}
        {children}
        <Footer /> {/* Server Component */}
      </body>
    </html>
  );
}

// app/navbar.tsx
export default function Navbar() {
  const user = await getSession(); // Server-side fetch
  return <header>Logged in as {user?.name}</header>;
}

// app/theme-toggle.tsx
'use client';
import { useState } from 'react';

export default function ThemeToggle() {
  const [theme, setTheme] = useState('light');
  return <button onClick={() => setTheme(theme === 'light' ? 'dark' :
'light')}>Toggle</button>;
}
```

Interview signal: "Server Components reduce JS to client; thoughtful boundary placement is key to performance."

---

3) Data Fetching Patterns and Caching Strategy

3.1 Server-Side Data Fetching (App Router)

Next.js provides built-in fetch caching:

```
// Automatic caching (default: 'force-cache')
const data = await fetch('https://api.example.com/posts');

// Opt-out of cache
const data = await fetch('https://api.example.com/live', {
```

```
  cache: 'no-store', // always fresh
});

// Revalidate after TTL
const data = await fetch('https://api.example.com/products', {
  next: { revalidate: 60 }, // cache for 60s, then revalidate
});

// Tags for on-demand revalidation
const data = await fetch('https://api.example.com/posts', {
  next: { tags: ['posts'] },
});
```

On-demand revalidation:

```
// app/api/revalidate/route.ts
export async function POST(request: Request) {
  const secret = request.headers.get('x-api-secret');
  if (secret !== process.env.REVALIDATE_SECRET) {
    return new Response('Unauthorized', { status: 401 });
  }
  const tag = new URL(request.url).searchParams.get('tag');
  revalidateTag(tag); // revalidate all routes with this tag
  return Response.json({ revalidated: true, now: Date.now() });
}
```

3.2 Client-Side Data Fetching with TanStack Query

For dynamic, client-driven data:

```
'use client';

import { useQuery } from '@tanstack/react-query';

export default function UserProfile() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['user', userId],
    queryFn: async () => {
      const res = await fetch(`/api/users/${userId}`);
      if (!res.ok) throw new Error('Failed to fetch');
      return res.json();
    },
    staleTime: 60000, // cache for 60s
    retry: 1,
  });

  if (isLoading) return <div>Loading...</div>;
  if (error) return <div>Error: {error.message}</div>;
  return <div>{data.name}</div>;
}
```

3.3 Cache Invalidation Patterns

Time-based: `revalidate` in `getStaticProps` or fetch `next: { revalidate: 60 }`.

Event-based: `revalidateTag()` or `revalidatePath()` on mutation.

```
// app/api/posts/route.ts
export async function POST(request: Request) {
  const { title, content } = await request.json();
  const post = await db.posts.create({ title, content });

  revalidateTag('posts'); // Invalidate all posts cache
  revalidatePath('/blog'); // Invalidate /blog page

  return Response.json(post);
}
```

Interview signal: "Cache invalidation is one of hardest problems; understand TTL, tags, and event-driven revalidation."

---

4) Performance Optimization and Core Web Vitals

4.1 Image Optimization (next/image)

- Built-in lazy loading, responsive sizing, format selection (WebP).
- Automatic srcset generation for multiple screen sizes.
- Prevents layout shift via aspect-ratio placeholder.

```
import Image from 'next/image';

export default function ProductImage({ src, alt }) {
  return (
    <Image
      src={src}
      alt={alt}
      width={300}
      height={300}
      priority={false} // lazy load by default
      placeholder="blur" // blur placeholder to prevent CLS
      sizes="(max-width: 768px) 100vw, 50vw"
      quality={80}
    />
  );
}
```

Interview trap: Using `<img>` instead of `<Image>` loses optimization; always use `next/image`.

4.2 Font Optimization (next/font)

- Self-hosted fonts via `@next/font`; no external requests, zero layout shift.

```tsx
import { Inter, Roboto } from 'next/font/google';

const inter = Inter({ subsets: ['latin'] });
const roboto = Roboto({ weight: ['400', '700'], subsets: ['latin'] });

// app/layout.tsx
export default function RootLayout({ children }) {
  return (
    <html className={inter.className}>
      <body>{children}</body>
    </html>
  );
}
```

4.3 Code Splitting and Bundle Analysis

```bash
# Analyze bundle
npm install --save-dev @next/bundle-analyzer
```

```tsx
// next.config.js
const withBundleAnalyzer = require('@next/bundle-analyzer')({
  enabled: process.env.ANALYZE === 'true',
});

module.exports = withBundleAnalyzer({
  // config
});
```

Run: `ANALYZE=true npm run build`

4.4 Dynamic Imports for Large Components

```tsx
import dynamic from 'next/dynamic';

const HeavyChart = dynamic(() => import('../components/HeavyChart'), {
  loading: () => <div>Loading chart...</div>,
  ssr: false, // Render on client only if necessary
});

export default function Dashboard() {
  return (
    <div>
      <HeavyChart />
    </div>
  );
}
```

4.5 Core Web Vitals Checklist

- **LCP (Largest Contentful Paint)**: Target < 2.5s. Optimize images, SSG, critical CSS.
- **FID (First Input Delay)**: Target < 100ms. Minimize JS, use web workers for heavy tasks.
- **CLS (Cumulative Layout Shift)**: Target < 0.1. Use Image priority, font-display: swap, aspect-ratio.

Interview signal: "Every optimization ties to business metrics; measure and monitor via real user data (RUM), not just synthetic."

---

5) API Routes and Edge Runtime

5.1 API Routes (Route Handlers in App Router)

```
// app/api/users/route.ts
export async function GET(request: Request) {
  const url = new URL(request.url);
  const id = url.searchParams.get('id');
  const user = await db.users.findOne({ id });
  return Response.json(user);
}

export async function POST(request: Request) {
  const body = await request.json();
  const user = await db.users.create(body);
  return Response.json(user, { status: 201 });
}
```

5.2 Middleware (Auth, Logging, Rate Limiting)

```
// middleware.ts
import { NextRequest, NextResponse } from 'next/server';

export function middleware(request: NextRequest) {
  const token = request.cookies.get('auth-token')?.value;

  if (!token && request.nextUrl.pathname.startsWith('/dashboard')) {
    return NextResponse.redirect(new URL('/login', request.url));
  }

  const response = NextResponse.next();
  response.headers.set('X-Request-ID', crypto.randomUUID());
  return response;
}

export const config = {
  matcher: ['/dashboard/:path*', '/api/:path*'],
};
```

5.3 Edge Runtime (Cloudflare Workers, Vercel Edge)

- Run code geographically closer to users; low latency, instant cold start.
- Constraints: no Node.js APIs (fs, net); use Web APIs.

```
// middleware.ts (runs on edge)
export function middleware(request: NextRequest) {
  if (request.geo?.country === 'US') {
    return NextResponse.rewrite(new URL('/us/home', request.url));
  }
  return NextResponse.next();
}

// app/api/edge-function/route.ts
export const runtime = 'edge';

export async function GET(request: Request) {
  const country = request.headers.get('cf-ipcountry') || 'Unknown';
  return new Response(`Served from edge to ${country}`);
}
```

Interview signal: "Edge runtime is powerful for latency-sensitive workloads (auth, redirects) but not suitable for CPU-heavy tasks."

---

6) Security Hardening and Best Practices

6.1 Environment Variables and Secrets

```
// .env.local (never commit; only NEXT_PUBLIC_* visible client-side)
DATABASE_URL=postgresql://...
API_SECRET=secret123
NEXT_PUBLIC_API_BASE=https://api.example.com

// Usage
const dbUrl = process.env.DATABASE_URL; // Server-side only
const apiBase = process.env.NEXT_PUBLIC_API_BASE; // Client-side accessible
```

Interview trap: Accidentally exposing secrets via `NEXT_PUBLIC_` or client bundle.

6.2 CSRF Protection

```
'use client';

import { useFormStatus } from 'react-dom';

export default function DeleteButton({ id }) {
  const { pending } = useFormStatus();

  return (
    <form action={deleteUser}>
      <input type="hidden" name="userId" value={id} />
```

```
      <button disabled={pending}>Delete</button>
    </form>
  );
}

// app/actions.ts (Server Action)
'use server';

export async function deleteUser(formData: FormData) {
  const userId = formData.get('userId');
  await db.users.delete({ id: userId });
  revalidatePath('/users');
}
```

CSRF tokens are auto-managed by Next.js Server Actions.

6.3 Input Validation and Sanitization

```
import { z } from 'zod';

const createUserSchema = z.object({
  email: z.string().email(),
  name: z.string().min(1).max(255),
  password: z.string().min(12),
});

export async function POST(request: Request) {
  const body = await request.json();

  try {
    const validated = createUserSchema.parse(body);
    const user = await db.users.create(validated);
    return Response.json(user, { status: 201 });
  } catch (error) {
    if (error instanceof z.ZodError) {
      return Response.json(error.errors, { status: 400 });
    }
    return Response.json({ error: 'Server error' }, { status: 500 });
  }
}
```

6.4 Authentication and Session Management

```
// lib/auth.ts
import { jwtVerify } from 'jose';

const secret = new TextEncoder().encode(process.env.JWT_SECRET!);

export async function verifyAuth(token: string) {
  try {
```

```
    const verified = await jwtVerify(token, secret);
    return verified.payload;
  } catch (err) {
    return null;
  }
}

// middleware.ts
export async function middleware(request: NextRequest) {
  const token = request.cookies.get('auth-token')?.value;
  const user = token ? await verifyAuth(token) : null;

  if (!user && request.nextUrl.pathname.startsWith('/dashboard')) {
    return NextResponse.redirect(new URL('/login', request.url));
  }

  const response = NextResponse.next();
  response.headers.set('X-User-ID', user?.id || '');
  return response;
}
```

6.5 Security Headers

```
// next.config.js
module.exports = {
  async headers() {
    return [
      {
        source: '/:path*',
        headers: [
          { key: 'X-Content-Type-Options', value: 'nosniff' },
          { key: 'X-Frame-Options', value: 'DENY' },
          { key: 'X-XSS-Protection', value: '1; mode=block' },
          { key: 'Strict-Transport-Security', value: 'max-age=31536000; includeSubDomains'
},
        ],
      },
    ];
  },
};
```

Interview signal: "Security isn't one thing; it's defense-in-depth: validation, secrets, auth, headers, CSP."

---

7) Deployment Strategies and Scaling

7.1 Vercel Deployment (Recommended for Next.js)

- Automatic preview deployments, edge caching, serverless functions.
- Integrated monitoring, analytics, performance optimizations.
- Zero-config setup; git-connected CI/CD.

```
npm i -g vercel
vercel deploy
```

7.2 Self-Hosted Deployment (Docker, Kubernetes)

```
# Dockerfile
FROM node:18-alpine AS base
WORKDIR /app
COPY package*.json ./
RUN npm ci

FROM base AS builder
COPY . .
RUN npm run build

FROM base AS runtime
COPY --from=builder /app/.next ./.next
COPY --from=builder /app/public ./public
ENV NODE_ENV=production
EXPOSE 3000
CMD ["npm", "start"]
```

Kubernetes deployment:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nextjs-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nextjs-app
  template:
    metadata:
      labels:
        app: nextjs-app
    spec:
      containers:
      - name: nextjs
        image: myregistry/nextjs-app:latest
        ports:
        - containerPort: 3000
        env:
        - name: DATABASE_URL
          valueFrom:
            secretKeyRef:
              name: app-secrets
              key: database-url
```

```
        livenessProbe:
          httpGet:
            path: /health
            port: 3000
          initialDelaySeconds: 10
          periodSeconds: 10
```

7.3 Performance Monitoring and Observability

```tsx
// app/layout.tsx
import { SpeedInsights } from "@vercel/speed-insights/next";
import { Analytics } from "@vercel/analytics/react";

export default function RootLayout({ children }) {
  return (
    <html>
      <body>
        {children}
        <Analytics /> {/* Captures Web Vitals */}
        <SpeedInsights /> {/* Real-time performance data */}
      </body>
    </html>
  );
}
```

Self-hosted with OpenTelemetry:

```tsx
import { nodeSDK } from '@opentelemetry/auto-instrumentations-node';
import { NodeSDK } from '@opentelemetry/sdk-node';

const sdk = new NodeSDK({
  traceExporter: new OTLPTraceExporter({ url: 'http://localhost:4317' }),
  instrumentations: [nodeSDK],
});

sdk.start();
```

Interview signal: "Monitoring production is non-negotiable; understand RUM vs synthetic, traces vs logs vs metrics."

---

8) Common Pitfalls and Incident Patterns

8.1 Hydration Mismatches

Problem: Server renders `<div>2024-01-01</div>` (from server date), client renders today's date.

```tsx
// WRONG: renders different dates on server vs client
export default function CurrentDate() {
  return <div>{new Date().toISOString()}</div>;
}
```

```
// RIGHT: suppress SSR or use client boundary
'use client';
import { useEffect, useState } from 'react';

export default function CurrentDate() {
  const [mounted, setMounted] = useState(false);
  useEffect(() => setMounted(true), []);

  if (!mounted) return <div>Loading...</div>;
  return <div>{new Date().toISOString()}</div>;
}
```

8.2 Slow Data Fetches Blocking Page Render

```
// SLOW: waits for all data before rendering
async function Page() {
  const user = await fetchUser(id);
  const posts = await fetchPosts(userId);
  const comments = await fetchComments(postsId);
  return <div>{/* render all */}</div>;
}

// FAST: parallelize + streaming
async function Page() {
  const userPromise = fetchUser(id);
  const postsPromise = fetchPosts(userId);

  const user = await userPromise;
  return (
    <div>
      <UserCard user={user} />
      <Suspense fallback={<div>Loading posts...</div>}>
        <PostsList postsPromise={postsPromise} />
      </Suspense>
    </div>
  );
}
```

8.3 Unbounded Memory Growth in Server Components

```
// WRONG: global cache grows without bound
let cache = {};

async function Page() {
  if (!cache[id]) {
    cache[id] = await fetchData(id);
  }
  return <div>{cache[id]}</div>;
```

```
}

// RIGHT: use external cache with TTL
export async function Page() {
  const data = await redis.get(`data:${id}`, async () => {
    return await fetchData(id);
  }, { ttl: 60 }); // 60s TTL
  return <div>{data}</div>;
}
```

8.4 Cache Stampede on Popular Content

```
// PROBLEM: thundering herd on cache expiry
// Multiple requests hit origin simultaneously after revalidate

// SOLUTION: stale-while-revalidate pattern
export async function Page() {
  const data = await fetch('https://api.example.com/trending', {
    next: {
      revalidate: 60, // cache for 60s
      tags: ['trending'],
    },
  });
  return <div>{data}</div>;
}

// Manually revalidate with debounce to avoid stampede
const revalidateDebounced = debounce(() => {
  revalidateTag('trending');
}, 5000); // coalesce requests within 5s window
```

8.5 Secrets Leaked in Build Output

```
# WRONG: secrets in environment during build
DATABASE_URL=postgresql://user:password@host/db npm run build

# RIGHT: secrets in runtime only
npm run build # No secrets during build
# At runtime (Docker, K8s, Vercel):
export DATABASE_URL=postgresql://...
npm start
```

Interview signal: "Incident patterns reveal production discipline; be ready to explain one incident you caused and fixed."

---

9) Production-Grade Architecture Patterns

9.1 Multi-Tenant SaaS with Next.js

```
// middleware.ts
export function middleware(request: NextRequest) {
  const { hostname } = request.nextUrl;
  const tenant = hostname.split('.')[0]; // Extract from subdomain

  const response = NextResponse.next();
  response.headers.set('X-Tenant-ID', tenant);
  return response;
}

// app/api/data/route.ts
export async function GET(request: Request) {
  const tenantId = request.headers.get('X-Tenant-ID');
  const data = await db.data.findMany({
    where: { tenantId },
  });
  return Response.json(data);
}
```

9.2 Feature Flags and A/B Testing

```
// lib/features.ts
import { getFlags } from '@vercel/flags';

export async function getFeatureFlags(userId?: string) {
  return getFlags(userId); // Returns flag overrides per user
}

// app/page.tsx
import { getFeatureFlags } from '@/lib/features';

export default async function Page() {
  const flags = await getFeatureFlags();

  return (
    <div>
      {flags.newCheckout ? <NewCheckout /> : <OldCheckout />}
    </div>
  );
}
```

9.3 Internationalization (i18n)

```
// middleware.ts
import { i18n } from './i18n.config';

export function middleware(request: NextRequest) {
  const pathname = request.nextUrl.pathname;
```

```
  const pathnameIsMissingLocale = i18n.locales.every(
    locale => !pathname.startsWith(`/${locale}/`) && pathname !== `/${locale}`
  );

  if (pathnameIsMissingLocale) {
    return NextResponse.redirect(
      new URL(`/en${pathname}`, request.url)
    );
  }
}

// app/[lang]/page.tsx
import { getDictionary } from '@/lib/i18n';

export default async function Page({ params: { lang } }) {
  const dict = await getDictionary(lang);
  return <h1>{dict.hello}</h1>;
}
```

9.4 Error Handling and Graceful Degradation

```
// app/error.tsx (Error Boundary)
'use client';

export default function Error({ error, reset }) {
  return (
    <div>
      <h2>Something went wrong: {error.message}</h2>
      <button onClick={() => reset()}>Try again</button>
    </div>
  );
}

// app/api/risky/route.ts
export async function GET(request: Request) {
  try {
    const data = await riskyOperation();
    return Response.json(data);
  } catch (error) {
    if (error instanceof TimeoutError) {
      return Response.json(
        { error: 'Operation timed out; please retry' },
        { status: 504 }
      );
    }
    return Response.json(
      { error: 'Internal server error' },
      { status: 500 }
    );
```

```
  }
}
```

---

10) L4/L5 Interview Grilling Questions

1. **How would you redesign homepage rendering for better Core Web Vitals?**
   - Analyze current metrics; shift to SSG where possible; lazy-load below-the-fold; optimize images; font-display: swap.
2. **Why did page load time increase after migration to App Router?**
   - Likely causes: too much Server Component work; unbounded data fetches; missing dynamic route caching; bundle not split properly.
3. **How would you handle cache invalidation at scale (millions of documents)?**
   - Use tags for logical grouping; event-driven invalidation from CMS; stale-while-revalidate to avoid stampedes; monitor revalidation latency.
4. **Design a checkout flow resilient to third-party payment provider outages.**
   - Offline-first local state; retry with exponential backoff; fallback payment method; queue failed payments for replay.
5. **How would you migrate from Pages Router to App Router without downtime?**
   - Phased migration; run both routers; feature flags for routing logic; canary new routes; monitor error rates and performance.
6. **What's your approach to monitoring and alerting in production Next.js?**
   - RUM for real-user data (Web Vitals, errors, custom metrics); synthetic monitoring for critical paths; log aggregation; SLO-based alerting.
7. **How would you prevent a slow API route from blocking other requests?**
   - Use queues for heavy tasks; offload to background jobs; stream responses; timeouts and circuit breakers.
8. **Design a recommendation engine frontend that scales to 10M users.**
   - ISR for popular recommendations; server-side personalization; cache by user segment; edge-compute personalization; fallback to global trends.
9. **Why can middleware latency impact user experience at global scale?**
   - Middleware runs on every request (auth, logging, redirects); even 50ms adds up at high traffic; use edge runtime for latency-critical logic.
10. **How would you approach a security incident where API keys were exposed in build logs?**
    - Immediately rotate keys; audit usage; add pre-deployment secret scanning; update documentation; run incident postmortem.

---

11) Production Story Prompts (7 YOE Context)

Prepare these with numbers and measurable outcomes:

- Reduced homepage TTFB from X ms to Y ms by migrating from SSR to ISR + edge caching.
- Prevented cache stampede incident by implementing smart cache warming and stale-while-revalidate.
- Fixed hydration mismatch bugs by introducing strict SSR/CSR boundaries with error monitoring.
- Scaled checkout conversion by introducing progressive rendering (Suspense + streaming).
- Migrated monolithic Pages Router to App Router incrementally over 6 weeks with zero downtime.
- Reduced API latency p99 by X% using rate limiting, timeouts, and circuit breakers.
- Implemented feature flags and A/B testing framework; shipped new UX with safe rollback.
- Led security audit; identified and patched X vulnerabilities; updated deployment process.

---

12) 90-Minute Pre-Interview Revision Sprint

0–30 min: Sections 1, 2, 4 (Rendering modes, Server/Client Components, Performance).

30–60 min: Sections 5, 6 (API Routes, Security).

60–90 min: Sections 8, 10 (Pitfalls, Interview Q&A, Story Rehearsal).

---

13) Architect-Level Checklist: Self-Assessment

- [ ] Can explain trade-offs of SSG vs SSR vs ISR vs CSR with real use cases.
- [ ] Understand Server Components mental model and when to use 'use client'.
- [ ] Know how to profile and optimize Core Web Vitals (LCP, FID, CLS).
- [ ] Can design multi-tenant SaaS architecture with Next.js.
- [ ] Understand cache invalidation strategies and their failure modes.
- [ ] Can talk through a security incident you've experienced.
- [ ] Know deployment options (Vercel, self-hosted, edge) and trade-offs.
- [ ] Can articulate why a specific rendering mode was chosen for a feature.
- [ ] Comfortable debugging hydration mismatches and performance regressions.
- [ ] Understand monitoring, observability, and SLO-driven alerting for production apps.

---

14) Common Architect Interview Structure

Problem Statement (5 min):

- "Design the homepage for an e-commerce site with 1M daily visitors."

Requirements Gathering (5 min):

- SEO needs, real-time inventory, user personalization, peak traffic patterns.

High-Level Design (5 min):

- Rendering strategy, caching tiers, data sources.

Deep Dive (10 min):

- Why SSG + ISR for product listings; why CSR for cart; cache invalidation on stock change.

Trade-offs (3 min):

- Build time vs freshness; cost vs performance; complexity vs maintainability.

Failure Handling (2 min):

- What if product API is down? Fallback to stale cache or generic page.

---

15) Key Takeaways for L4/L5 Interviews

- **Rendering is not one-size-fits-all**: Choose based on audience (SEO), freshness, personalization, and scale.
- **Server Components reduce JS to client**: Thoughtful SSR/CSR boundaries are a core architectural skill.
- **Performance is measured, not guessed**: Use Web Vitals, RUM, and synthetic monitoring to drive decisions.
- **Cache invalidation is hard**: Understand TTL, tags, event-driven revalidation, and failure modes.
- **Security is defense-in-depth**: Validate inputs, manage secrets, use CSRF tokens, set headers, rotate keys.
- **Observability wins production incidents**: Logs, traces, metrics, and SLO-based alerting separate good systems from great ones.

Interview signal: "I don't just use Next.js; I reason about its design, trade-offs, failure modes, and how it scales to millions of users."

---

16) Source Guidance for This Playbook

This playbook synthesizes patterns and concepts from:

- Next.js official documentation (nextjs.org).
- Vercel engineering blog and performance case studies.
- React server component patterns (react.dev).
- Web Vitals and CWV best practices (web.dev).
- Security guidelines from OWASP and CWE.
- Community-driven production patterns (Rauchg, Kent C. Dodds, etc.).